

SKILL WEEK 5

Task 1: Change Directory

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

To change the current working directory, validate the given path, maintain the previous directory, and handle invalid paths.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char current[PATH_MAX];
    char previous[PATH_MAX];
    char path[PATH_MAX];

    if (getcwd(current, sizeof(current)) == NULL) {
        perror("getcwd");
        return 1;
    }

    strcpy(previous, current);

    printf("Current Directory: %s\n", current);

    printf("Enter directory to change to: ");
    fgets(path, sizeof(path), stdin);
    path[strcspn(path, "\n")] = '\0';

    if (chdir(path) == 0) {
        printf("Directory changed successfully.\n");

        getcwd(current, sizeof(current));

        printf("Previous Directory: %s\n", previous);
        printf("Current Directory: %s\n", current);
    } else {
        perror("Error changing directory");
    }

    return 0;
}
```

Input

/home/amrutha

Output

```
amrutha@blue:~$ mkdir -p SKILL_Week5
amrutha@blue:~$ cd SKILL_Week5
amrutha@blue:~/SKILL_Week5$ pwd
/home/amrutha/SKILL_Week5
amrutha@blue:~/SKILL_Week5$ nano cd.c
amrutha@blue:~/SKILL_Week5$ gcc cd.c -o cd
amrutha@blue:~/SKILL_Week5$ ./cd
Current Directory: /home/amrutha/SKILL_Week5
Enter directory to change to: /home/amrutha
Directory changed successfully.
Previous Directory: /home/amrutha/SKILL_Week5
Current Directory: /home/amrutha
```

amrutha@blue:~/OSSP/SKILL_Week5\$ gcc cd.c -o cd

amrutha@blue:~/OSSP/SKILL_Week5\$./cd

Current Directory: /home/amrutha/OSSP/SKILL_Week5

Enter directory to change to: /home/amrutha

Directory changed successfully.

Previous Directory: /home/amrutha/OSSP/SKILL_Week5

Current Directory: /home/amrutha

Test Invalid Path

Run:

cd ~/OSSP/SKILL_Week5

./cd

Input:

/home/amrutha/abc

Output:

```
amrutha@blue:~/SKILL_Week5$ cd ~/OSSP/SKILL_Week5
-bash: cd: /home/amrutha/OSSP/SKILL_Week5: No such file or directory
amrutha@blue:~/SKILL_Week5$ ./cd
Current Directory: /home/amrutha/SKILL_Week5
Enter directory to change to: /home/amrutha/abc
Error changing directory: No such file or directory
```

Current Directory: /home/amrutha/OSSP/SKILL_Week5

Enter directory to change to: /home/amrutha/abc

Error changing directory: No such file or directory

Observation

The given directory path was validated using `chdir()`. A valid path changed the working directory successfully, while an invalid path generated an error.

Result

Directory navigation was successfully implemented with previous and current directory tracking.

Task 2: Built-in Command Dispatch

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

To identify built-in shell commands and execute them directly using a command dispatch table.

Program

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>

void cmd_pwd() {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void cmd_cd(char *path) {
    if (path == NULL || strlen(path) == 0) {
        printf("cd: missing argument\n");
    }
    else if (chdir(path) != 0) {
        perror("cd");
    }
}

void cmd_exit() {
    printf("Exiting shell...\n");
    exit(0);
}
```

```

int main() {
    char input[100];

    printf("Built-in Shell\n");

    while (1) {
        printf("myShell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "pwd") == 0) {
            cmd_pwd();
        }
        else if (strncmp(input, "cd ", 3) == 0) {
            cmd_cd(input + 3);
        }
        else if (strcmp(input, "exit") == 0) {
            cmd_exit();
        }
        else if (strlen(input) == 0) {
            continue;
        }
        else {
            printf("Invalid command: %s\n", input);
        }
    }

    return 0;
}

```

Terminal Commands

```

amrutha@blue:~/SKILL_Week5$ nano builtin.c
amrutha@blue:~/SKILL_Week5$ gcc builtin.c -o builtin
amrutha@blue:~/SKILL_Week5$ ./builtin

```

Input

Enter the commands one by one:

pwd

cd /home

pwd

hello

exit

Output

```
Built-in Shell
myShell> pwd
/home/amrutha/SKILL_Week5
myShell> cd /home
myShell> pwd
/home
myShell> hello
Invalid command: hello
myShell> exit
Exiting shell...
```

Observation

The shell identified built-in commands such as `pwd`, `cd`, and `exit`. Invalid commands were detected and reported without terminating the shell.

Result

Built-in command dispatch was successfully implemented and tested.

Task 3: `pwd` and `exit`

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

To retrieve and display the current working directory and safely process the exit command.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char cwd[1024];
    char command[50];

    while (1) {
        printf("myShell> ");

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "pwd") == 0) {
```

```

        if (getcwd(cwd, sizeof(cwd)) != NULL)
            printf("Current Directory: %s\n", cwd);
        else
            perror("getcwd");
    }

    else if (strcmp(command, "exit") == 0) {

        printf("Saving shell state...\n");
        printf("Cleaning up resources...\n");
        printf("Shell exited successfully.\n");

        exit(0);
    }

    else if (strlen(command) == 0) {
        continue;
    }

    else {
        printf("Invalid command.\n");
    }
}

return 0;
}

```

Input

pwd

Then:

hello

Then:

exit

Output

```

amrutha@blue:~/SKILL_Week5$ nano pwd_exit.c
amrutha@blue:~/SKILL_Week5$ gcc pwd_exit.c -o pwd_exit
amrutha@blue:~/SKILL_Week5$ ./pwd_exit
myShell> pwd
Current Directory: /home/amrutha/SKILL_Week5
myShell> hello
Invalid command.
myShell> exit
Saving shell state...
Cleaning up resources...
Shell exited successfully.

```

Observation

The pwd command successfully retrieved the current working directory using getcwd(). The exit command performed the required cleanup messages before terminating the program.

Result

The current directory was displayed successfully and the shell exited correctly when requested.

Task 4

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

To parse the export command, validate environment variable names, create or update variables, and verify the stored value.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int valid_name(char *name) {
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main() {
    char input[200];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strncmp(input, "export ", 7) != 0) {
        printf("Invalid command.\n");
        return 1;
    }

    char *assignment = input + 7;
    char *equal = strchr(assignment, '=');
```

```

    if (equal == NULL) {
        printf("Syntax Error: Expected NAME=VALUE\n");
        return 1;
    }

    *equal = '\0';

    char *name = assignment;
    char *value = equal + 1;

    if (!valid_name(name)) {
        printf("Invalid variable name.\n");
        return 1;
    }

    if (setenv(name, value, 1) == 0) {
        printf("Environment variable updated successfully.\n");
        printf("%s=%s\n", name, getenv(name));
    }
    else {
        perror("setenv");
    }

    return 0;
}

```

Output

```

amrutha@blue:~/SKILL_Week5$ nano export.c
amrutha@blue:~/SKILL_Week5$ gcc export.c -o export
amrutha@blue:~/SKILL_Week5$ ./export
myShell> export NAME=Amrutha
Environment variable updated successfully.
NAME=Amrutha
amrutha@blue:~/SKILL_Week5$ ./export
myShell> export CITY=Hyderabad
Environment variable updated successfully.
CITY=Hyderabad
amrutha@blue:~/SKILL_Week5$ ./export
myShell> export 123NAME=Hello
Invalid variable name.
amrutha@blue:~/SKILL_Week5$ ./export
myShell> export NAME
Syntax Error: Expected NAME=VALUE
amrutha@blue:~/SKILL_Week5$ |

```

Observation

The export command was parsed into a variable name and value. Valid variable names were accepted and updated using setenv(), while invalid names and incorrect syntax were rejected.

Result

Environment variable creation and updating were successfully implemented and tested.